

APOSTILA DIDÁTICA

Desenvolvimento Web

Aula 01 — Fundamentos da Web e Ecossistema Digital

Análise e Desenvolvimento de Sistemas — 2º Semestre
Módulo 1: Fundamentos da Web e Estruturação Semântica
Prof. Anderson Pazin

1. História e Evolução da Web

A World Wide Web que conhecemos hoje é resultado de décadas de pesquisa, inovação e colaboração global. Para compreender o desenvolvimento web moderno, é fundamental conhecer as origens e a evolução dessa tecnologia que transformou a sociedade.

1.1 As Origens: ARPANET e a Internet

A história começa em 1969, quando a DARPA (*Defense Advanced Research Projects Agency*) dos Estados Unidos criou a ARPANET — a primeira rede de computadores do mundo. O objetivo inicial era criar uma rede de comunicação descentralizada que pudesse sobreviver a ataques nucleares, conectando universidades e centros de pesquisa.

Linha do Tempo — Marcos Históricos

- **1969 — ARPANET:** Primeira rede de computadores criada pelo DARPA (EUA). Conectou inicialmente 4 universidades: UCLA, Stanford, UC Santa Barbara e University of Utah.
- **1983 — TCP/IP:** Adoção do protocolo TCP/IP como padrão de comunicação. Este é o "idioma" que todos os computadores na internet falam até hoje.
- **1989 — Tim Berners-Lee propõe a WWW:** Físico britânico trabalhando no CERN (Suíça) propõe um sistema de gerenciamento de informações baseado em hipertexto para facilitar o compartilhamento de dados entre pesquisadores.
- **1991 — Primeiro site do mundo:** O site `info.cern.ch` é publicado — ainda acessível hoje. Continha documentação sobre o projeto WWW e como criar páginas web.
- **1993 — Navegador Mosaic:** Primeiro navegador gráfico popular, desenvolvido pela NCSA. Permitia exibir imagens junto ao texto, tornando a web visualmente acessível ao público geral.
- **1994 — Fundação do W3C:** Tim Berners-Lee funda o *World Wide Web Consortium* (W3C) para padronizar as tecnologias da web (HTML, CSS, etc.) e garantir interoperabilidade entre navegadores.

1.2 As Gerações da Web

A evolução da web pode ser dividida em gerações, cada uma com características tecnológicas e sociais distintas:

Geração	Período	Características	Tecnologias	Exemplos
Web 1.0	1991–2000	Somente leitura. Páginas estáticas. Conteúdo publicado por poucos, consumido por muitos.	HTML básico, FTP, e-mail	Sites institucionais simples
Web 2.0	2001–2015	Leitura e escrita. Interatividade. Conteúdo gerado pelo usuário. Redes sociais.	AJAX, JavaScript, CSS3, APIs REST	Facebook (2004), YouTube (2005), Twitter (2006)
Web 3.0	2015–hoje	Web semântica, IA, IoT, descentralização. Dados conectados e experiências personalizadas.	Blockchain, IA/ML, WebAssembly, PWA	ChatGPT, DeFi, NFTs, assistentes virtuais

 **Conceito-Chave:** A transição da Web 1.0 para a Web 2.0 foi marcada pela democratização da criação de conteúdo. Qualquer pessoa passou a poder publicar, comentar e interagir — não apenas consumir. Isso transformou a web de uma "biblioteca digital" em uma "praça pública global".

2. Internet vs. Web

Um dos conceitos mais importantes — e mais frequentemente confundidos — é a diferença entre Internet e Web. Eles não são sinônimos!

- **Internet:** É a infraestrutura física e lógica de rede global. Inclui cabos submarinos, roteadores, servidores, protocolos TCP/IP. É o "encanamento" que conecta bilhões de dispositivos.
- **Web (WWW):** É um serviço que roda sobre a Internet, usando os protocolos HTTP/HTTPS e HTML para exibir páginas e hiperlinks. É apenas uma das aplicações da Internet.
- **E-mail:** Outro serviço que roda sobre a Internet, usando os protocolos SMTP, POP3 e IMAP. Não é parte da Web.
- **Jogos Online / FTP / VoIP:** Outros serviços que usam a Internet como infraestrutura, mas não fazem parte da Web (não usam HTTP/HTML).

 **Analogia:** Pense na Internet como as rodovias e estradas do país. A Web seria como os carros que trafegam por essas estradas. O e-mail seria como os ônibus. Todos usam a mesma infraestrutura (estradas/Internet), mas são serviços diferentes.

3. Arquitetura Cliente-Servidor

Toda comunicação na web segue o modelo Cliente-Servidor. Neste modelo, há dois papéis distintos: o cliente (quem solicita) e o servidor (quem responde).

3.1 Os Papéis

- **Cliente (Browser):** Qualquer dispositivo que solicita recursos: navegadores web, aplicativos mobile, ferramentas de linha de comando (curl, wget). Inicia sempre a comunicação.
- **Servidor:** Computador que armazena e serve recursos (HTML, CSS, JS, imagens, dados). Aguarda requisições e responde com os recursos solicitados ou mensagens de erro.

3.2 Fluxo Completo de uma Requisição Web

Quando você digita <https://www.google.com> no navegador, o seguinte processo ocorre:

1. **Digitação da URL:** O usuário digita a URL no navegador e pressiona Enter.
2. **Consulta ao DNS:** O navegador consulta o DNS para traduzir www.google.com em um endereço IP (ex: 142.250.74.46). Verifica cache local → cache do SO → servidor DNS do ISP → servidores raiz.
3. **Estabelecimento de Conexão TCP:** O navegador estabelece uma conexão TCP com o servidor no IP retornado pelo DNS, na porta 80 (HTTP) ou 443 (HTTPS). Para HTTPS, ocorre também o handshake TLS.
4. **Envio da Requisição HTTP:** O navegador envia uma HTTP Request ao servidor com o método (GET), o caminho (/), a versão do protocolo e os headers (User-Agent, Accept, etc.).
5. **Processamento no Servidor:** O servidor recebe a requisição, processa (busca o arquivo HTML, executa código, consulta banco de dados) e prepara a resposta.
6. **Resposta HTTP:** O servidor envia uma HTTP Response com o status code (200 OK), headers de resposta e o corpo (HTML, JSON, imagem, etc.).
7. **Renderização pelo Navegador:** O navegador interpreta o HTML, faz novas requisições para CSS, JS e imagens referenciados, e renderiza a página visualmente para o usuário.

✓ **Dica Prática:** Você pode observar todo esse processo na aba Network do Chrome DevTools (F12). Cada linha representa uma requisição HTTP separada — uma para o HTML, outras para CSS, JS, imagens, fontes, etc.

4. URLs e DNS

4.1 Anatomia de uma URL

Uma URL (Uniform Resource Locator) é o endereço completo de um recurso na web. Cada parte tem um significado específico:

```
https://www.exemplo.com.br:443/produtos/camiseta?cor=azul&tamanho=M#detalhes
```

https	→ Protocolo (esquema de comunicação)
www	→ Subdomínio
exemplo.com.br	→ Domínio (nome do site)
:443	→ Porta (443 = HTTPS padrão, 80 = HTTP padrão)
/produtos/camiseta	→ Caminho (path) do recurso
?cor=azul&tamanho=M	→ Query string (parâmetros)
#detalhes	→ Fragmento (âncora na página)

4.2 DNS — O Catálogo Telefônico da Internet

O DNS (*Domain Name System*) é um sistema hierárquico distribuído que traduz nomes de domínio legíveis por humanos (google.com) em endereços IP numéricos (142.250.74.46) que os computadores usam para se comunicar.

Processo de Resolução DNS

1. **Cache do Navegador:** O browser verifica se já resolveu esse domínio recentemente e tem o IP em cache.
2. **Cache do Sistema Operacional:** O SO verifica seu próprio cache DNS e o arquivo /etc/hosts.
3. **Resolver do ISP:** O servidor DNS do seu provedor de internet (ISP) é consultado. Ele também tem cache próprio.
4. **Servidores Raiz (Root DNS):** 13 grupos de servidores raiz no mundo que conhecem os servidores autoritativos para cada TLD (.com, .br, .org).
5. **Servidor TLD:** O servidor do TLD (.com, .com.br) indica qual é o servidor autoritativo para o domínio específico.
6. **Servidor Autoritativo:** Retorna o endereço IP definitivo do domínio. O resultado é cacheado em todos os níveis para futuras consultas.

5. Protocolo HTTP e HTTPS

5.1 HTTP vs. HTTPS

Característica	HTTP	HTTPS
Nome completo	HyperText Transfer Protocol	HTTP Secure
Porta padrão	80	443
Criptografia	Nenhuma — dados em texto puro	TLS/SSL — dados criptografados
Certificado	Não necessário	Certificado digital obrigatório
Segurança	Vulnerável a interceptação (MITM)	Protegido contra interceptação
SEO	Penalizado pelo Google	Favorecido pelo Google
Uso recomendado	Apenas desenvolvimento local	Todos os sites em produção

⚠️ Atenção! Nunca insira dados sensíveis (senhas, cartão de crédito, CPF) em sites que usam HTTP (sem o "S"). Os dados trafegam em texto puro e podem ser interceptados por qualquer pessoa na mesma rede.

5.2 Estrutura de uma Requisição HTTP

```
GET /produtos/camiseta?cor=azul HTTP/1.1
Host: www.exemplo.com.br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: pt-BR,pt;q=0.9,en;q=0.8
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: session_id=abc123; preferencias=tema_escuro
```

5.3 Estrutura de uma Resposta HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 4823
Date: Wed, 30 Jul 2026 12:00:00 GMT
Server: nginx/1.24.0
Cache-Control: max-age=3600
Set-Cookie: session_id=xyz789; HttpOnly; Secure; SameSite=Strict

<!DOCTYPE html>
<html lang="pt-BR">
  <!-- Conteúdo HTML da página -->
</html>
```

5.4 Métodos HTTP

Os métodos HTTP definem a intenção da requisição — o que o cliente quer fazer com o recurso:

Método	Ação	Idempotente?	Uso Típico
GET	Recuperar um recurso	Sim	Buscar página, listar produtos
POST	Criar novo recurso	Não	Enviar formulário, criar usuário
PUT	Atualizar recurso completo	Sim	Atualizar perfil completo
DELETE	Remover um recurso	Sim	Excluir comentário, deletar conta
PATCH	Atualização parcial	Não	Atualizar apenas o status de um pedido

5.5 Códigos de Status HTTP

Categoria	Código	Significado	Exemplo de Uso
1xx — Informacional	100 Continue	Servidor recebeu parte da requisição	Upload de arquivo grande
2xx — Sucesso	200 OK	Requisição bem-sucedida	Página carregada com sucesso
2xx — Sucesso	201 Created	Recurso criado com sucesso	Usuário cadastrado
2xx — Sucesso	204 No Content	Sucesso sem corpo de resposta	DELETE bem-sucedido
3xx — Redirecionamento	301 Moved Permanently	Recurso movido permanentemente	Migração de domínio
3xx — Redirecionamento	304 Not Modified	Recurso não foi alterado (cache)	Recurso em cache válido
4xx — Erro do Cliente	400 Bad Request	Requisição malformada	JSON inválido no body
4xx — Erro do Cliente	401 Unauthorized	Autenticação necessária	Token JWT ausente
4xx — Erro do Cliente	403 Forbidden	Acesso negado	Usuário sem permissão
4xx — Erro do Cliente	404 Not Found	Recurso não encontrado	Página inexistente
5xx — Erro do Servidor	500 Internal Server Error	Erro genérico no servidor	Bug no código back-end
5xx — Erro do Servidor	503 Service Unavailable	Servidor indisponível	Manutenção ou sobrecarga

6. Navegadores e Servidores Web

6.1 Navegadores Web

Um navegador web (browser) é uma aplicação que interpreta e renderiza HTML, CSS e JavaScript para exibir páginas web ao usuário. Cada navegador usa um motor de renderização diferente:

Navegador	Motor de Renderização	Market Share (2025)	Desenvolvedor
Google Chrome	Blink	~65%	Google
Apple Safari	WebKit	~19%	Apple
Microsoft Edge	Blink (Chromium)	~5%	Microsoft
Mozilla Firefox	Gecko	~3%	Mozilla Foundation
Opera	Blink (Chromium)	~2%	Opera Software

6.2 Servidores Web

Um servidor web é um software que recebe requisições HTTP e retorna respostas. Ele serve arquivos estáticos (HTML, CSS, imagens) e pode delegar requisições dinâmicas para servidores de aplicação.

Servidor	Market Share	Características	Uso Típico
Nginx	~34%	Alta performance, baixo consumo de memória, excelente como proxy reverso	Sites de alto tráfego, CDN, proxy reverso
Apache HTTP Server	~30%	Modular, configurável via .htaccess, amplamente documentado	Hospedagem compartilhada, sites PHP
Cloudflare Server	~22%	CDN integrado, proteção DDoS, edge computing	Sites com necessidade de CDN global
Node.js (Express)	Crescente	JavaScript no servidor, event-driven, não-bloqueante	APIs REST, aplicações em tempo real

i Servidor Web vs. Servidor de Aplicação: Servidor Web (Nginx, Apache): serve arquivos estáticos e encaminha requisições dinâmicas. Servidor de Aplicação (Node.js, Tomcat, Gunicorn): executa código de negócio, acessa banco de dados, gera respostas dinâmicas. Em produção, geralmente usam-se ambos em conjunto.

7. Ambiente de Desenvolvimento

7.1 Visual Studio Code

O VS Code (Visual Studio Code) é o editor de código mais popular do mundo, usado por 73% dos desenvolvedores (Stack Overflow Developer Survey 2023). É gratuito, open-source, desenvolvido pela Microsoft e disponível para Windows, macOS e Linux.

Configurações Recomendadas

Configuração	Valor Recomendado	Justificativa
Auto Save	afterDelay	Salva automaticamente, permitindo que o Live Server atualize em tempo real
Tab Size	2	Padrão da comunidade web (HTML/CSS/JS). Economiza espaço horizontal.
Word Wrap	on	Quebra linhas longas, evitando scroll horizontal
Format On Save	true	Formata o código automaticamente ao salvar (requer Prettier)

7.2 Extensões Essenciais

- **Live Server:** Cria um servidor local e atualiza o navegador automaticamente a cada salvamento. Essencial para desenvolvimento web. Autor: Ritwick Dey.
- **Prettier:** Formatador de código automático para HTML, CSS e JavaScript. Garante consistência no estilo do código em todo o projeto.
- **Auto Rename Tag:** Renomeia automaticamente a tag de fechamento quando você altera a tag de abertura.
- **Color Highlight:** Exibe uma prévia visual das cores diretamente no código CSS. Facilita o trabalho com paletas de cores.

7.3 Emmet — Produtividade HTML/CSS

O Emmet é uma ferramenta nativa do VS Code que permite escrever HTML e CSS muito mais rapidamente usando abreviações. Basta digitar a abreviação e pressionar Tab:

```
! + Tab      → Gera estrutura HTML5 completa
div.container + Tab → <div class="container"></div>
ul>li*5 + Tab → <ul> com 5 <li> filhos
h1+p+p + Tab → h1 seguido de 2 parágrafos
a[href="#" ] + Tab → <a href="#"></a>
table>tr*3>td*4 → Tabela 3x4 completa
```

7.4 Atalhos Essenciais do VS Code

Ação	Windows/Linux	macOS
Paleta de comandos	Ctrl+Shift+P	Cmd+Shift+P
Comentar/descomentar linha	Ctrl+/	Cmd+/
Abrir terminal integrado	Ctrl+` (crase)	Cmd+` (crase)
Mover linha para cima/baixo	Alt+↑/↓	Option+↑/↓
Duplicar linha abaixo	Shift+Alt+↓	Shift+Option+↓
Selecionar próxima ocorrência	Ctrl+D	Cmd+D
Formatar documento	Shift+Alt+F	Shift+Option+F
Ir para linha específica	Ctrl+G	Cmd+G

8. DevTools — Aba Network

O Chrome DevTools é um conjunto de ferramentas de desenvolvimento integrado ao navegador. A aba Network é fundamental para entender como as páginas web carregam e como o protocolo HTTP funciona na prática.

8.1 Abrindo o DevTools

- **F12** — Abre o DevTools (atalho principal)

- **Ctrl+Shift+I** — Abre o DevTools (alternativo)
- **Ctrl+Shift+J** — Abre o DevTools na aba Console
- **Ctrl+Shift+C** — Abre o DevTools no modo inspeção de elemento

8.2 Funcionalidades da Aba Network

- **Gravação de Requisições:** Registra automaticamente todas as requisições HTTP feitas pela página. Recarregue a página com o DevTools aberto para capturar tudo.
- **Filtros por Tipo:** Filtre requisições por tipo: All, XHR, JS, CSS, Img, Media, Font, Doc, WS, Wasm, Manifest, Other.
- **Inspeção Detalhada:** Clique em qualquer requisição para ver: Headers (cabeçalhos), Payload (dados enviados), Preview, Response (resposta bruta), Timing e Cookies.
- **Throttling de Rede:** Simule conexões lentas (3G, Slow 3G) ou offline para testar a performance da página em condições adversas.

8.3 Colunas Principais da Tabela de Requisições

Coluna	Descrição	Exemplo
Name	Nome do arquivo ou endpoint requisitado	index.html, style.css, api/users
Status	Código de status HTTP da resposta	200, 304, 404, 500
Type	Tipo MIME do recurso retornado	document, stylesheet, script, xhr
Initiator	O que causou a requisição	parser (HTML), script.js:42
Size	Tamanho da resposta (comprimida/descomprimida)	4.2 kB / 12.8 kB
Time	Tempo total da requisição	45 ms, 1.2 s
Waterfall	Visualização gráfica do timing da requisição	Barra colorida mostrando DNS, TCP, TTFB, Download

 **Dica Profissional:** Marque a opção "Disable cache" na aba Network para garantir que todas as requisições sejam feitas ao servidor durante os testes, sem usar versões em cache. Isso é essencial para testar mudanças em CSS e JS.

9. Atividade Prática — Encontro 2

 **Missão:** Criar seu primeiro site e inspecionar as requisições HTTP geradas. Objetivo: conectar a teoria de HTTP e cliente-servidor com a prática real do desenvolvimento web.

Parte 1 — Setup do Ambiente de Desenvolvimento

1. **Instalar o VS Code:** Acesse code.visualstudio.com, clique em "Download for [seu SO]" e siga o instalador. No Windows, marque "Adicionar ao PATH" e "Open with Code" no menu de contexto.

2. **Instalar extensões essenciais:** Abra o VS Code → Ctrl+Shift+X → instale: Live Server (Ritwick Dey), Prettier - Code formatter e Auto Rename Tag.
3. **Configurar o VS Code:** Acesse File → Preferences → Settings (ou Ctrl+,) e configure: Auto Save → afterDelay, Tab Size → 2, Word Wrap → on, Format On Save → true.
4. **Criar a estrutura do projeto:** Crie uma pasta chamada aula01-meu-site em um local de fácil acesso. Abra-a no VS Code: File → Open Folder.
5. **Criar o arquivo index.html com Emmet:** No Explorer do VS Code, clique no ícone "New File" e nomeie index.html. No arquivo vazio, digite apenas ! e pressione Tab.

Parte 2 — Construindo o Conteúdo HTML

Substitua o conteúdo do <body> pelo código abaixo:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Meu Primeiro Site — Aula 01</title>
</head>
<body>
  <!-- Cabeçalho principal da página -->
  <h1>Fundamentos da Web 🌐</h1>
  <p>Página criada na Aula 01 do curso de Desenvolvimento Web.</p>

  <h2>O que aprendi hoje</h2>
  <ul>
    <li>A web foi criada por Tim Berners-Lee em 1989 no CERN</li>
    <li>HTTP é o protocolo de comunicação da web (porta 80)</li>
    <li>HTTPS adiciona criptografia TLS/SSL (porta 443)</li>
    <li>DNS traduz nomes de domínio em endereços IP</li>
    <li>O navegador é o cliente; o servidor responde às requisições</li>
  </ul>

  <h2>Métodos HTTP</h2>
  <ol>
    <li><strong>GET</strong> — Recuperar um recurso</li>
    <li><strong>POST</strong> — Enviar dados para criar recurso</li>
    <li><strong>PUT</strong> — Atualizar recurso completo</li>
    <li><strong>DELETE</strong> — Remover um recurso</li>
  </ol>

  <h2>Referências</h2>
  <p>Saiba mais em: <a href="https://developer.mozilla.org/pt-BR">MDN Web Docs</a></p>
</body>
</html>
```

Parte 3 — Servidor Local com Live Server

1. **Iniciar o Live Server:** Com o arquivo index.html aberto, clique no botão "Go Live" na barra de status inferior do VS Code. O navegador abrirá automaticamente em <http://127.0.0.1:5500/index.html>.

2. **Testar o hot-reload:** Volte ao VS Code, altere o texto do `<h1>` para qualquer coisa e salve (Ctrl+S). Observe o navegador atualizar automaticamente sem precisar recarregar manualmente.
3. **Entender o que acontece:** O Live Server cria um servidor HTTP local na porta 5500. Quando você acessa `http://127.0.0.1:5500`, seu navegador (cliente) faz uma requisição HTTP GET ao servidor local, que responde com o arquivo HTML.

Parte 4 — Inspeção HTTP com Chrome DevTools

1. **Abrir o DevTools na aba Network:** No Chrome, pressione F12 para abrir o DevTools. Clique na aba Network.
2. **Desabilitar o cache e recarregar:** Marque a opção "Disable cache". Pressione Ctrl+Shift+R (hard reload). Observe as requisições aparecendo na tabela em tempo real.
3. **Inspeccionar a requisição principal:** Clique na primeira linha da tabela (o arquivo `index.html`). Um painel lateral abrirá com as abas: Headers, Preview, Response e Timing.
4. **Analisar os Headers:** Na aba Headers, observe: Request Headers (método GET, User-Agent, Accept) e Response Headers (Status 200 OK, Content-Type: text/html, Date).
5. **Registrar suas observações:** Anote: Qual foi o método HTTP? Qual o status code? Qual o Content-Type? Quanto tempo levou a requisição? Quantas requisições foram feitas no total?

10. Exercícios Práticos e Atividades Hands-On

Bloco A — Desafios de Configuração do Ambiente

Desafio A1 — Configuração Completa do VS Code

- Instale as 3 extensões obrigatórias: Live Server, Prettier e Auto Rename Tag. Tire um screenshot da aba Extensions mostrando as 3 como "Installed".
- Configure o VS Code com Auto Save (afterDelay), Tab Size 2 e Format On Save habilitado. Abra o arquivo `settings.json` (Ctrl+Shift+P → "Open User Settings JSON") e mostre as configurações aplicadas.
- Crie um arquivo `teste.html`, use o Emmet (! + Tab) para gerar a estrutura, adicione um `<h1>` com seu nome e abra com o Live Server. Confirme que o navegador exibe seu nome.
- **Desafio extra:** Instale também a extensão Color Highlight e abra um arquivo CSS com cores hexadecimais. Observe as bolinhas coloridas ao lado dos valores.

Desafio A2 — Produtividade com Emmet

Pratique os atalhos Emmet abaixo. Para cada um, escreva a abreviação, pressione Tab e observe o HTML gerado:

- Digite `div.container>h1+p*3` + Tab. Qual estrutura HTML foi gerada? Quantas tags foram criadas?
- Digite `ul>li.item$*5` + Tab. O que o símbolo \$ faz? Quais classes foram geradas?

- Digite `a[href="https://mdn.mozilla.org"] {Documentação MDN}` + Tab. Qual o resultado?
- Digite `table>tr*3>td*4` + Tab. Quantas células foram criadas no total?
- **Desafio:** Crie com Emmet em uma única linha uma estrutura com: um `<nav>` contendo uma `<ul>` com 4 `<li>`, cada um com um `<a>` com `href="#"`.

Desafio A3 — Atalhos de Teclado Essenciais

Execute cada ação usando apenas o teclado (sem mouse) e registre o atalho utilizado:

Ação	Atalho (Windows/Linux)	Atalho (macOS)
Abrir paleta de comandos	Ctrl+Shift+P	Cmd+Shift+P
Comentar/descomentar linha	Ctrl+/	Cmd+/
Abrir terminal integrado	Ctrl+` (crase)	Cmd+` (crase)
Mover linha para cima	Alt+↑	Option+↑
Duplicar linha abaixo	Shift+Alt+↓	Shift+Option+↓
Selecionar próxima ocorrência	Ctrl+D	Cmd+D
Formatar documento	Shift+Alt+F	Shift+Option+F
Ir para linha específica	Ctrl+G	Cmd+G

Bloco B — Exercícios de Inspeção HTTP com DevTools

Desafio B1 — Análise de Requisições em Site Real

- Acesse www.uol.com.br com o DevTools aberto na aba Network. Responda: Quantas requisições foram feitas no total? Qual o tamanho total transferido? Quanto tempo levou o carregamento completo?
- Filtre as requisições por tipo Doc. Qual é o status code da requisição principal? Qual o Content-Type retornado?
- Filtre por JS. Quantos arquivos JavaScript foram carregados? Qual o maior deles?
- Filtre por CSS. Quantas folhas de estilo foram carregadas? Alguma retornou status 304 (Not Modified)?
- Clique em qualquer requisição XHR (se houver). O que é uma requisição XHR? Para que ela é usada?

Desafio B2 — Comparando HTTP vs HTTPS na Prática

- Acesse <http://neverssl.com> (site que usa HTTP puro) com o DevTools aberto. Observe a barra de endereços: aparece algum aviso de segurança? Qual o status code da requisição principal?
- Agora acesse <https://www.google.com>. Compare: qual a diferença visual na barra de endereços? Qual a porta usada (veja nos Headers)?

- No DevTools do Google, clique na requisição principal e observe os Response Headers. Você consegue identificar algum header relacionado à segurança (ex: Strict-Transport-Security)?
- **Reflexão:** Por que um site de banco NUNCA deve usar HTTP? O que poderia acontecer se usasse?

Desafio B3 — Simulando Conexão Lenta

- Acesse qualquer site com imagens (ex: globo.com) com o DevTools aberto na aba Network.
- No seletor de throttling (dropdown que mostra "No throttling"), selecione "Slow 3G". Recarregue a página (Ctrl+Shift+R).
- Observe: Quanto tempo levou agora? Quais recursos demoram mais para carregar? As imagens aparecem progressivamente?
- Selecione "Offline" e tente recarregar. Qual erro aparece? Qual o status code?
- **Reflexão:** Por que é importante testar sites em conexões lentas? Quem são os usuários que podem ter esse problema no Brasil?

Desafio B4 — Lendo Headers HTTP Manualmente

Analise os headers abaixo (de uma requisição real) e responda as perguntas:

```
HTTP/2 200 OK
content-type: text/html; charset=UTF-8
content-encoding: gzip
cache-control: max-age=3600, public
last-modified: Wed, 30 Jul 2026 08:00:00 GMT
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: SAMEORIGIN
server: nginx/1.24.0
content-length: 4823
```

- Qual o tipo de conteúdo retornado? Qual o charset (codificação de caracteres)?
- O que significa **content-encoding: gzip**? Por que isso é usado?
- O que significa **cache-control: max-age=3600**? Por quanto tempo o browser pode usar a versão em cache?
- O que é o header **strict-transport-security**? Qual sua função de segurança?
- Qual servidor web está sendo usado? Qual versão?

Bloco C — Questões Conceituais sobre Cliente-Servidor e Protocolos

Questões C1 — Múltipla Escolha

1. Qual das afirmações abaixo está CORRETA sobre a diferença entre Internet e Web?

- (a) Internet e Web são sinônimos
- (b) A Web é a infraestrutura física; a Internet é o serviço de páginas HTML
- (c) A Internet é a infraestrutura de rede; a Web é um serviço que roda sobre ela usando HTTP
- (d) A Web existia antes da Internet, criada por Tim Berners-Lee em 1969

2. No modelo Cliente-Servidor, qual afirmação é VERDADEIRA?

- (a) O servidor sempre inicia a comunicação enviando dados ao cliente
- (b) O cliente sempre inicia a comunicação fazendo uma requisição ao servidor
- (c) Cliente e servidor se comunicam simultaneamente sem ordem definida
- (d) O DNS é o servidor e o navegador é o cliente em toda comunicação web

3. Qual método HTTP deve ser usado para DELETAR um recurso em uma API REST?

- (a) GET
- (b) POST
- (c) PUT
- (d) DELETE

4. Um usuário acessa um site e recebe o status code 404. O que isso significa?

- (a) O servidor está fora do ar
- (b) O usuário não tem permissão para acessar o recurso
- (c) O recurso solicitado não foi encontrado no servidor
- (d) A requisição foi bem-sucedida e o recurso foi criado

5. Qual porta padrão o protocolo HTTPS utiliza?

- (a) 21
- (b) 80
- (c) 443
- (d) 8080

Questões C2 — Verdadeiro ou Falso (Justifique)

Para cada afirmação, indique V (Verdadeiro) ou F (Falso) e escreva uma justificativa de 1-2 frases:

- () O DNS é necessário apenas para sites com HTTPS; sites HTTP não precisam de DNS.
- () Uma URL pode conter protocolo, domínio, porta, caminho, query string e fragmento.
- () O método GET pode ser usado para enviar dados sensíveis como senhas, pois os dados ficam no corpo da requisição.
- () O status code 301 indica que um recurso foi movido permanentemente para outro endereço.
- () O Live Server cria um servidor HTTP local, simulando o comportamento de um servidor web real.
- () O Chrome DevTools só funciona em sites HTTPS; não é possível inspecionar requisições HTTP.
- () Tim Berners-Lee criou a Internet em 1969 ao desenvolver a ARPANET no DARPA.
- () O header Content-Type: text/html informa ao navegador como interpretar o corpo da resposta.

Questões C3 — Dissertativas

1. **Questão 1 (Fluxo completo):** Descreva detalhadamente, passo a passo, o que acontece quando um usuário digita `https://www.ifsp.edu.br` no navegador e pressiona Enter. Inclua:

- resolução DNS, estabelecimento de conexão TCP, handshake TLS, envio da requisição HTTP, processamento no servidor e renderização no navegador.
2. **Questão 2 (Análise comparativa):** Compare as três gerações da web (1.0, 2.0 e 3.0) em termos de: (a) tipo de interação do usuário, (b) tecnologias predominantes, (c) exemplos de aplicações e (d) modelo de negócio. Organize sua resposta em uma tabela.
 3. **Questão 3 (Cenário prático):** Você é desenvolvedor de um e-commerce. Um cliente relata que ao tentar finalizar uma compra, a página trava e não processa o pagamento. Descreva como você usaria o Chrome DevTools (aba Network) para diagnosticar o problema. Quais informações você buscaria? Quais status codes indicariam diferentes tipos de problema?
 4. **Questão 4 (Segurança):** Uma empresa pequena diz que não precisa de HTTPS porque "não tem dados sensíveis, só um site institucional com texto e imagens". Você concorda? Elabore uma argumentação técnica explicando por que TODOS os sites devem usar HTTPS, mesmo os puramente informativos.

Bloco D — Mini-Projeto Guiado: Página de Perfil Profissional

★ **Sobre este Mini-Projeto:** Você vai criar uma página de perfil profissional completa, aplicando todos os conceitos da Aula 01. Este projeto será a base do seu Projeto Integrador da disciplina — nas próximas aulas, você adicionará CSS, JavaScript e mais funcionalidades.

Objetivo do Projeto

Criar uma página HTML semântica que funcione como um cartão de visitas digital, contendo: informações pessoais, habilidades, projetos e links de contato. A página deve ser acessível via Live Server e ter suas requisições inspecionadas no DevTools.

Estrutura de Arquivos do Projeto

```
meu-perfil/  
├── index.html      # Página principal (você vai criar)  
├── sobre.html      # Página "Sobre mim" (opcional)  
├── assets/  
│   └── imagens/    # Pasta para imagens futuras
```

Passo 1 — Criar a Estrutura Base

Crie a pasta meu-perfil, abra no VS Code e crie o index.html com Emmet. Altere o <title> para seu nome:

```
<!DOCTYPE html>  
<html lang="pt-BR">  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <meta name="description" content="Perfil profissional de [Seu Nome] —  
Estudante de ADS">  
    <title>[Seu Nome] — Perfil Profissional</title>  
  </head>  
  <body>  
    <!-- Conteúdo será adicionado nos próximos passos -->  
  </body>  
</html>
```

Passo 2 — Seção de Cabeçalho (Header)

```
<!-- CABEÇALHO: Apresentação pessoal -->
<header>
  <h1>[Seu Nome Completo]</h1>
  <p>Estudante de Análise e Desenvolvimento de Sistemas</p>
  <p>📍 [Sua Cidade], Brasil</p>
  <nav>
    <a href="#sobre">Sobre</a> |
    <a href="#habilidades">Habilidades</a> |
    <a href="#projetos">Projetos</a> |
    <a href="#contato">Contato</a>
  </nav>
</header>
```

Passo 3 — Seção "Sobre Mim"

```
<main>
  <section id="sobre">
    <h2>Sobre Mim</h2>
    <p>
      Olá! Sou estudante do curso de Análise e Desenvolvimento de
      Sistemas.
      Tenho interesse em [suas áreas de interesse] e estou aprendendo
      desenvolvimento web neste semestre.
    </p>
  </section>
```

Passo 4 — Seção de Habilidades com Lista

```
<section id="habilidades">
  <h2>Habilidades em Desenvolvimento</h2>
  <h3>Tecnologias que estou aprendendo:</h3>
  <ul>
    <li>HTML5 – Estruturação semântica de páginas web</li>
    <li>CSS3 – Estilização e layout responsivo</li>
    <li>JavaScript – Interatividade e lógica no front-end</li>
    <li>VS Code – Editor de código com extensões</li>
  </ul>
  <h3>Conceitos que já domino:</h3>
  <ol>
    <li>Protocolo HTTP/HTTPS e métodos (GET, POST, PUT,
    DELETE)</li>
    <li>Arquitetura Cliente-Servidor e fluxo de requisições</li>
    <li>DNS e estrutura de URLs</li>
    <li>Inspeção de requisições com Chrome DevTools</li>
  </ol>
</section>
```

Passo 5 — Seção de Projetos com Tabela

```
<section id="projetos">
  <h2>Meus Projetos</h2>
  <table>
    <caption>Projetos desenvolvidos no curso de ADS</caption>
    <thead>
      <tr>
        <th>Projeto</th>
        <th>Tecnologias</th>
        <th>Status</th>
      </tr>
    </thead>
  </table>
```

```

        <th>Link</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Página de Perfil</td>
        <td>HTML5</td>
        <td>Em andamento</td>
        <td><a href="#">Ver projeto</a></td>
      </tr>
    </tbody>
  </table>
</section>

```

Passo 6 — Seção de Contato e Rodapé

```

<section id="contato">
  <h2>Entre em Contato</h2>
  <ul>
    <li>✉ E-mail: <a href="mailto:seuemail@exemplo.com">seuemail@exemplo.com</a></li>
    <li>🔗 LinkedIn: <a href="https://linkedin.com/in/seuperfil" target="_blank">linkedin.com/in/seuperfil</a></li>
    <li>🐙 GitHub: <a href="https://github.com/seuusuario" target="_blank">github.com/seuusuario</a></li>
  </ul>
</section>
</main>

<footer>
  <p>© 2026 [Seu Nome] — Desenvolvido na disciplina de
  <strong>Desenvolvimento Web</strong></p>
</footer>

```

Passo 7 — Validação e Inspeção Final

1. Abrir com Live Server e verificar se todos os links de navegação (#sobre, #habilidades, etc.) funcionam corretamente ao clicar.
2. Inspecionar no DevTools (F12) → aba Network → recarregar. Quantas requisições foram feitas? Apenas 1 (o HTML) ou mais? Por quê?
3. Validar o HTML: Acesse validator.w3.org, cole seu HTML e verifique se há erros. Corrija qualquer erro ou aviso encontrado.
4. Personalizar e expandir: Substitua todos os placeholders com suas informações reais. Adicione pelo menos mais 2 habilidades e 1 projeto à tabela.

✅ **Critérios de Avaliação do Mini-Projeto:** (1) Estrutura HTML correta com todas as seções solicitadas; (2) Uso adequado de tags semânticas (<header>, <main>, <section>, <footer>, <nav>); (3) Pelo menos uma lista ordenada, uma não-ordenada e uma tabela; (4) Links funcionais (internos e externos); (5) Validação sem erros no W3C Validator; (6) Screenshot do DevTools mostrando a requisição HTTP da página.

11. Referências e Leituras Complementares

- **MDN Web Docs — Como a Web funciona:** developer.mozilla.org/pt-BR/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works
- **MDN Web Docs — Visão geral do HTTP:** developer.mozilla.org/pt-BR/docs/Web/HTTP/Overview
- **MDN Web Docs — Métodos de requisição HTTP:** developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods
- **Chrome DevTools — Aba Network:** developer.chrome.com/docs/devtools/network/overview
- **W3C — História da Web:** [w3.org/History.html](https://www.w3.org/History.html)
- **VS Code — Documentação Oficial:** code.visualstudio.com/docs
- **Emmet — Cheat Sheet:** docs.emmet.io/cheat-sheet
- **W3C Markup Validation Service:** validator.w3.org
- **GeeksforGeeks — History and Evolution of Web Development:** [geeksforgeeks.org/blogs/history-and-evolution-of-web-development](https://www.geeksforgeeks.org/blogs/history-and-evolution-of-web-development)
- **AWS — Diferença entre HTTP e HTTPS:** aws.amazon.com/pt/compare/the-difference-between-https-and-http

*Apostila elaborada para a disciplina de Desenvolvimento Web — ADS 2º Semestre | 2026 | Referências:
MDN Web Docs · Chrome for Developers · W3C · GeeksforGeeks · AWS*